Chapter 3. Strings and Things

3.0 Introduction

Character strings are an inevitable part of just about any programming task. We use them for printing messages for the user; for referring to files on disk or other external media; and for people's names, addresses, and affiliations. The uses of strings are many, almost without number (actually, if you need numbers, we'll get to them in Chapter 5).

If you're coming from a programming language that is at or below C level, you'll need to remember that `String` is a defined type (class) in Java—that is, a string is an object and therefore has methods. It is not an array of characters (though it contains one) and should not be thought of as an array. Operations like `fileName.endsWith(".gif")` and `extension.equals(".gif")` (and the almost-equivalent `".gif".equals(extension)`) are commonplace.[1]

Java old-timers should note that Java 11 and 12 added several new `String` methods, including `indent(int n)`, `stripLeading()` / `stripTrailing()`, `Stream<String> lines()`, `isBlank()`, `repeat(int n)`, and `transform()`. Java 15 added `stripIndent()`. Most of these provide obvious functionality; `transform` allows applying an instance of a functional interface (see 9.0 Introduction) to a string and returning the result of that operation.

Although we haven't discussed the details of the java.io package yet (we will, in Chapter 10), you need to be able to read text files for some of these programs. Even if you're not familiar with `java.io`, you can probably see from the examples of reading text files that a `BufferedReader` allows you to read chunks of data and that this class has a very convenient `readLine()` method.

Going the other way, `System.out.println()` is normally used to print strings or other values to the terminal or standard output. String concatenation is commonly used here, like this:

```
System.out.println("The answer is " + result);
```

One caveat with string concatenation is that if you are appending a bunch of values, and a number and a `char` (single character) are concatenated together, they are *added before concatenation* due to Java's precedence rules. So don't do as I did in this contrived example:

```
int result = ...;
System.out.println(result + '=' + " the answer."); // FAILURE
```

Given that `result` is an integer, then `result + =` (`result` added to the equals sign character, which is of the numeric type `char`) is a valid *numeric* expression, which will result in a single value of type `int` . If the variable `result` has the value 42, and given that the character `=` in a Unicode (or ASCII) code chart has the value 61, the two-line fragment would print:

```
103 the answer.
```

The wrong value and no equals sign! Safer approaches include using parentheses, double quotes around the equals sign, a `StringBuilder` (see Recipe 3.3), or `String.format()` (see Recipe 3.2). Of course in this simple example you could just move the `=` to be part of the string literal, but the example was chosen to illustrate the problem of arithmetic on `char` values being confused with string contatenation.

I won't show you how to sort an array of strings here; the more general notion of sorting a collection of objects will be taken up in Recipe 7.11.

String literals are *interned*, that is, they are stored internally. Interning is the process of storing only one copy of each unique string value in a pool of strings, as part of the class file containing the string. When a `String` is created at runtime, Java first checks if an equivalent string is present in the pool; if so, the existing string reference is returned instead of creating a new `String` object. This saves memory by avoiding duplicate string objects with the same value. The file *main/src/main/java/strings/Equality.java* demonstrates the results of interning, and a method of requesting it.

14 Java 14 enables text blocks, also known as multiline text strings. These are delimited with a set of three double quotes, the opening of which *must* have a newline after the quotes (which doesn't become part of the string; the following newlines do):

```
String longStr = """
This is a long
```

```
text String.""";
```

If we print `longStr`, we get:

```
This is a long
  text String.
```

TIMELESS, IMMUTABLE, AND UNCHANGEABLE

Notice that a given `String` object, once constructed, is immutable. In other words, once I have said `String s = "Hello" + yourName;`, the contents of the particular object that reference variable `s` refers to can never be changed. You can assign `s` to refer to a different string, even one derived from the original, as in `s = s.trim()`. And you can retrieve characters from the original string using `charAt()`, but it isn't called `getCharAt()` because there is not, and never will be, a `setCharAt()` method. Even methods like `toUpperCase()` don't change the `String`; they return a new `String` object containing the translated characters. If you need to change characters within a `String`, you should instead create a `StringBuilder` (possibly initialized to the starting value of the `String`), manipulate the `StringBuilder` to your heart's content, and then convert that to `String` at the end, using the ubiquitous `toString()` method.[2]

How can I be so sure they won't add a `setCharAt()` method in the next release? Because the immutability of strings is one of the fundamentals of the Java Virtual Machine (JVM). Immutable objects are generally good for software reliability (some languages do not even allow mutable objects). Immutability avoids conflicts, particularly where multiple threads are involved, or where software from multiple organizations has to work together; for example, you can safely pass immutable objects to a third-party library and expect that the objects will not be modifed.

It may be possible to tinker with the `String`'s internal data structures using native code, as shown in Recipe 18.6, but then all bets are off.

Remember also that the `String` is a fundamental type in Java. Unlike most of the other classes in the core API, the behavior of strings is not changeable by subclassing; the class is marked `final` so it cannot be subclassed. So you can't declare your own `String` subclass. Think if you could—you could masquerade as a `String` but provide a `setCharAt()` method! Again, they thought of that. If you don't believe me, try it out:

```
public class WolfInStringsClothing
  extends java.lang.String {    //EXPECT COMPILE ERROR
  private static final long serialVersionUID = 1;

  public void setCharAt(int index, char newChar) {
    // The implementation of this method
    // would be left as an exercise for the reader.
    // Hint: compile this code exactly as is before bothering!
  }
}
```

Got it? They thought of that!

Of course you do need to be able to modify strings. Some methods extract part of a `String`; these are covered in the first few recipes in this chapter. And `StringBuilder` is an important class that deals in characters and strings and has many methods for changing the contents, including, of course, a `toString()` method. Reformed C programmers should note that Java strings are not arrays of `char`s as in C. Therefore, you must use methods for such operations as processing a string one character at a time; see Recipe 3.4. Figure 3-1 shows an overview of `String`, `StringBuilder`, and C-language strings.

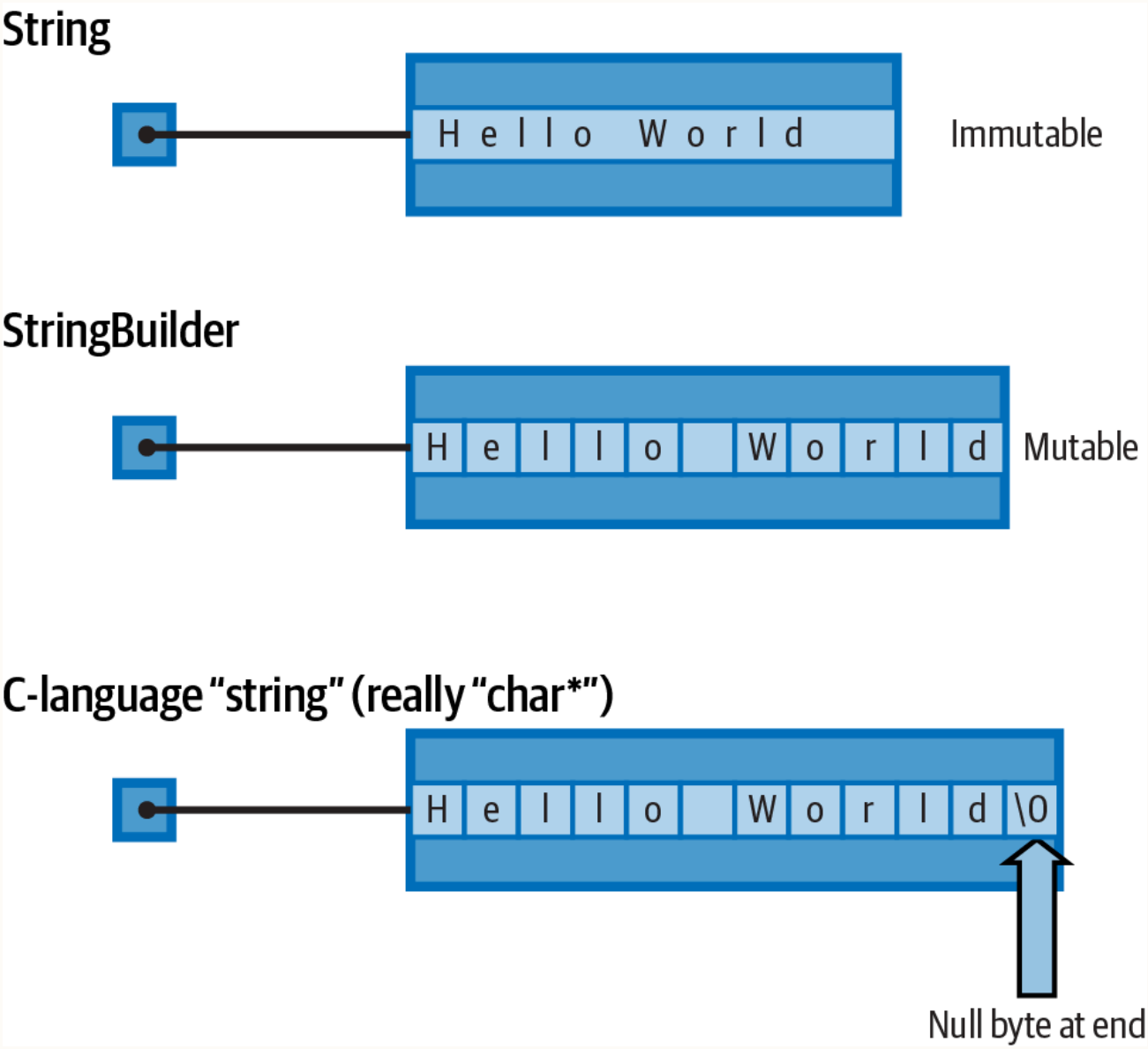


Figure 3-1. `String`, `StringBuilder`, and C-language strings

3.1 Taking Strings Apart with Substrings,

Tokenizing, and Trimming Methods

Problem

You want to break a string apart, either by indexing positions or by using fixed token characters (e.g., break on spaces to get words).

Solution

For substrings, use the `String` object's `substring()` method. For tokenizing, construct a `StringTokenizer` around your string and call its methods `hasMoreTokens()` and `nextToken()`.

Or, use regular expressions (see Chapter 4).

Discussion

We'll look first at substrings, and then discuss tokenizing.

Substrings

The `substring()` method constructs a new `String` object made up of a run of characters contained somewhere in the original string, the one whose `substring()` you called. The `substring()` method is overloaded: both forms require a starting index (which is always *zero-based*). The one-argument form returns from `startIndex` to the end. The two-argument form takes an ending index (not a length, as in some languages) so that an index can be generated by the `String` methods `indexOf()` or `lastIndexOf()`:

```
public class SubStringDemo {
  public static void main(String[] av) {
    String a = "Java is great.";
    System.out.println(a);
    String b = a.substring(5);  // b is the String "is great."
    System.out.println(b);
    String c = a.substring(5,7);// c is the String "is"
    System.out.println(c);
    String d = a.substring(5,a.length());// d is "is great."
    System.out.println(d);
  }
}
```

When run, this prints the following:

```
C:> java SubStringDemo.java
Java is great.
is great.
is
is great.
C:>
```

WARNING

Note that the end index is one beyond the last character! Java adopts this policy of having a half-open interval (or inclusive start, exclusive end) fairly consistently. There are good practical reasons for adopting this approach, and some other languages do so too.

Tokenizing

One of the easiest ways to split a string into words is to use the `String.split()` method. You can easily split a string into words like this:

```
for (String word : "Hello World".split(" ")) {
   System.out.println(word);
}
Hello
World
```

The parameter is actually a regular expression, discussed in Chapter 4. As a result, you can match multiple spaces, or spaces and tabs, by using the string `"\s+"`.

If you want to split the lines in a CSV file, you could try the string `","` but it's surely safer to use one of several third-party libraries for CSV files (because of issues like escaped commas in quoted string values).

Another method is to use a `StringTokenizer`. The `StringTokenizer` methods implement the `Iterator` interface and design pattern (see Recipe 7.7). However, its usage is officially "discouraged," but you will find it used in existing code:

main/src/main/java/strings/StrTokDemo.java

```
StringTokenizer st = new StringTokenizer("Hello World of Java");

while (st.hasMoreTokens( ))
   System.out.println("Token: " + st.nextToken( ));
```

The `split()` method mentioned previously is usually a better choice.

Sometimes you don't need to take the string completely apart, but simply remove leading or trailing blanks. For this, use the `String` class `strip()` or `trim()` methods.

There are five methods in the `String` class for this:

`String trim()`

: Returns the string with all leading and trailing space characters removed

`strip()`

: Returns the string with all leading and trailing whitespace removed

`stripLeading()`

: Returns the string with all leading whitespace removed

`stripTrailing()`

: Returns the string with all trailing whitespace removed

`stripIndent()`

: Returns the multiline string with indendation minimized

For the `trim()` method, space includes any character whose numeric value is less than or equal to 32, 0x20, or U+0020 (the actual space character). For the `strip()` methods, whitespace is as defined by `Character.isSpace()`, i.e., spaces, tabs, form feeds, etc.

See Also

Many occurrences of `StringTokenizer` can be replaced with regular expressions (see Chapter 4) with considerably more flexibility. For example, to extract all the numbers from a `String`, you can use this code:

```
Matcher tokenizer = Pattern.compile("\\d+").matcher(inputString);
while (tokenizer.find( )) {
    String courseString = tokenizer.group(0);
    int courseNumber = Integer.parseInt(courseString);
    ... do computation with courseNumber, or just print it ...
```

This allows user input to be more flexible than you could easily handle with a `StringTokenizer`. Assuming that the numbers represent course numbers at some educational institution, the inputs "471,472,570," or "Courses 471 and 472, 570," or just "471 472 570" all give the same results.

3.2 String Formatting with Formatter and printf()

Problem

You want to have fine-grained control over formatting values into strings, in such contexts as:

- The static `String.format()` method
- The Java 15 `string.formatted()` instance method
- The `printf()` method in the `PrintStream` and `PrintWriter` classes

Solution

Use the formatting capabilities of the `java.util.Formatter` class, which is used internally by at least the APIs listed previously.[3]

Discussion

The `java.util.Formatter` class uses format codes derived from the C library's `printf()` function.[4] `PrintStream` and `PrintWriter` have convenience routines named `printf()` that simply delegate to the stream or writer's `format()` method, which in turn delegates to a `Formatter` instance. Unlike C, however, Java is a strongly typed language, so invalid arguments will throw an exception rather than generating gibberish. There are also convenience routines `static String.format()` and instance `string.formatted()` for use when you want to format a `String` without creating a `Formatter` explicitly and/or don't want it sent directly to an output stream or writer. You can even create a `Formatter` instance

with a filename; this overload returns a `BufferedWriter` whose `format()` methods will write into the file. Close that when done, or use in a `try-with-resources` to automatically close it.

TIP

There are many classes to perform formatting in Java. Do not confuse `java.util.Formatter` with `java.text.Format`. `Formatter` is for `printf`-like formatting of one or more values with control over formatting. `Format` is a base class with subclasses for formatting dates, strings, numbers, lists, and more, all in a locale-sensitive way. `Format` has a subclass `MessageFormat`, which uses curly-brace markers to interpolate parameters into a string. `Format` is also the base class of my `StringAlign` class (see Recipe 3.5) and my `RomanNumberFormat` class (discussed in a subsection of Recipe 5.5).

The underlying `java.util.Formatter` class works on a "format string" containing format codes. For each item that you want to format, you put a format code into the format string, and pass a parameter to be formattted according to that code. The format code consists of a percent sign, optionally an argument number followed by a dollar sign, optionally a field width or precision, and a format type (e.g., `d` for decimal integer, that is, an integer with no decimal point, and `f` for floating point).

One of the simplest forms is when you just need to format a string:

```
var str = String.format("Hello %s, today is %tF%n", name, now);
```

15 Some developers (and some IDEs) prefer to provide a string containing the format codes, using the instance method `formatted`. The following provides the same formatting result as the previous code:

```
var str2 = "Hello %s, today is %tF%n".formatted(name, now);
```

This form is useful in internationalization (see Recipe 3.11) when you want to provide different strings for use in different locales. However, it is *strongly* recommended to avoid using user-provided format strings—those you might have read in from a user, especially over the internet—since the language that `format()` accepts is such a broad attack surface from a security point of view.

In any JDK in living memory, a simple usage that combines formatting with output might look like the following:

```
// To output the formatted result:
System.out.printf("%04d - the year of %f%n", 1956, Math.PI);
// or, more formally:
System.out.printf("1$%04d - the year of %2$f%n", 1956, Math.PI);
```

As shown in Figure 3-2, the `"%04d"` or `"%1$04d"` controls formatting of the year, and the `"%f"` or `"%2$f"` controls formatting of the value of `PI`.[5]

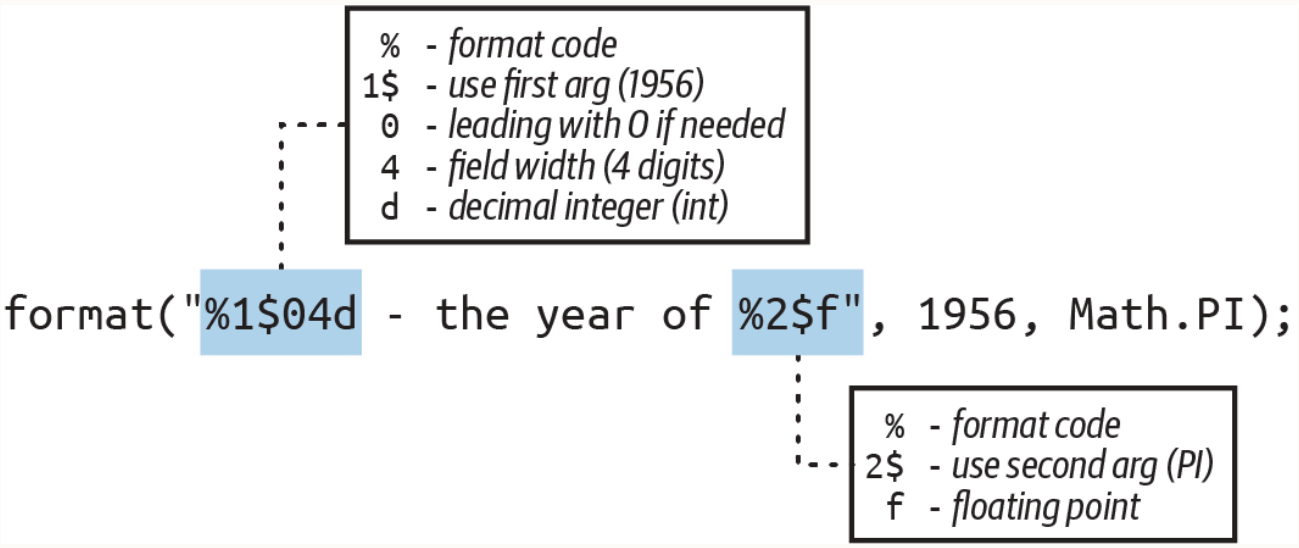


Figure 3-2. `Formatter` codes examined

Many format codes are available; Table 3-1 lists some of the more common ones. There is considerably more detail and more choices in the Javadoc for `java.util.Formatter`.

Table 3-1. `Formatter` format codes

Code	Meaning
`b`	Boolean true or false.
`c`	Character (argument must be `char` or integral type containing valid character value).
`d`	decimal int—integer to be printed as a decimal (radix 10) with no decimal point (argument must be integral type).
`e`	Floating-point value printed in scientific ("exponential") notation.
`f`	Floating-point value with decimal fraction (must be numeric); field width may be followed by decimal point and fractional digit field width; e.g., 7.2f.
`g`	Floating-point value, as per f or e, depending on magnitude.
`h`	Print object's `hashCode()` in hex, as per `Integer.toHexString(arg.hashCode())`.
`s`	Generic format; if value is null, prints `null`; else if `arg` implements `Formattable`, format as per `arg.formatTo()`; else format as per `arg.toString()`.
`t`	Date codes; follow with secondary code. Common date codes are shown in Table 3-2. Argument must be `long`, `Long`, `LocalDate` and friends, or the legacy `Calendar` or `java.util.Date`.
`n`	Newline; insert the platform-dependent line ending character.
`%`	Insert a literal `%` character.

The letters `b`, `c`, `e`, `h`, `s` may be entered in uppercase, in which case the result will be capitalized as appropriate.

Note also that you may, but are not required to, put a *parameter order* number between the `%` and the format code. For example, in `"%2$04d"`, the `"2$"` means to format the *second* parameter, regardless of the order of the parameters. This is primarily useful with dates (see the following example, where you need to format several different portions of the same `Date` or `Calendar`, or any time you want to format the same object more than once) and in internationalization, where different languages may require words to be in a different order within a sentence.

Some examples of using a `Formatter` are shown in Example 3-1.

Example 3-1. main/src/main/java/io/FormatterDemo.java

```
public class FormatterDemo {
  public static void main(String[] args) throws IOException {

    // The arguments to all these format methods consist of
    // a format code String and 1 or more arguments.
    // Each format code consists of the following:
    // % - code lead-in
    // N$ - OPTIONAL parameter number (1-based) after the format code
    // N - field width
    // L - format letter (d: decimal(int); f: float; s: general; many more)

    // Most general (cumbersome) way of proceeding.
    Formatter fmtr = new Formatter();
    // The format() method returns a Formatter, so use Object
    Object result = fmtr.format("%1$04d - the year of %2$f", 1956, Math.PI);
    System.out.println(result);

    // Shorter way using static String.format(), default parameter numbering.
    String stringResult = "%04d - the year of %f".formatted(1956, Math.PI);
    System.out.println(stringResult);

    // A shorter way using PrintStream/PrintWriter.format, more in line with
    // other languages. Using this way, you should provide the newline delimiter
    // using %n (rather than \n as that is platform-dependent!).
    System.out.printf("%04d - the year of %f%n", 1956, Math.PI);

    // Format doubles with more control
    System.out.printf("PI is approximately %4.2f%n", Math.PI);

    final String TMPFILE = "/tmp/format-demo.txt";
    Formatter fmt2 = new Formatter(TMPFILE);
    fmt2.format(
      "This is a %s opportunity that you can get in for just $%5.2f!%n",
      "new", 27.99);
    fmt2.close();

    String result2 = Files.readString(Path.of(TMPFILE));
    System.out.print("String from file is: " + result2);
  }
}
```

Running `FormatterDemo` produces this:

```
C:> java FormatterDemo.java
1956 - the year of 3.141593
1956 - the year of 3.141593
1956 - the year of 3.141593
PI is approximately 3.14
String from file is: This is a new opportunity that you can get in for just $27.99!
C:>
```

For formatting date and time objects, a large variety of format codes is available—about 40 in all. Date and time objects are discussed in Chapter 6. Table 3-2 shows the more common date/time format codes. Each must be preceded by a `t`, so to format the first argument as a year, you would use `%1$tY`.

Table 3-2. Formatting codes for dates and times

Format code	Meaning
`A`	Locale-specific day of week (`a` for abbreviated)
B	Locale-specific month name (`b` for abbreviated)
`d`	Day of month (two digits, leading zeros)
e	Day of month (one or two digits)
`H` or `I`	Hour in 24-hour (`H`) or 12-hour (`I`) format (two digits, leading zero if needed)
M	Minute (two digits)
`m`	Month as two-digit (leading zeros) number
P / p	Locale-specific AM or PM in uppercase (if `P`) or lowercase (if `p`)
`S`	Second (two digits)
y	Two-digit year (for those who don't remember Y2K)
`Y`	Year (at least four digits)

There are also some composite date codes, including those in Table 3-3.

Table 3-3. Composite formatting codes for dates and times

Format code	Meaning
F	ISO 8601 standard date formatted as `"%tY-%tm-%td"`
D	US-only date format as `"%tm/%td/%ty"`
R or T	24-hour time combination: `%tH:%tM` (if `R`) or `%tH:%tM:%tS` (if `T`)
c	Fancy date string, requires `java.util.Date` or `ZonedDateTime`, not `LocalDateTime`

In my opinion, embedding the codes from Table 3-2 directly into applications that you distribute or make available as web applications is often a bad idea. Any direct use of them assumes that you know *the correct order to print these fields in all locales around the world*. Trust me, you don't. Instead of these, I recommend the use of `DateTimeFormatter`, covered in Recipe 6.2, to control the order of arguments. However, for quick-and-dirty work, as well as for writing log- or datafiles that must be in a given format because some other program reads them, these are OK. Some date examples are shown in Example 3-2.

Example 3-2. main/src/main/java/io/FormatterDates.java

```
public class FormatterDates {
  public static void main(String[] args) {

    // Format number as dates e.g., 2026-06-28
    System.out.printf("%4d-%02d-%2d%n", 2026, 6, 28);

    // Format fields directly from a Date object: multiple fields from "1$"
    // (hard-coded formatting for Date not advisable; see I/O chapter)
    LocalDate today = LocalDate.now();
    // Print in a form like e.g., "July 4, 2026"
    System.out.printf("Today is %1$tB %1$td, %1$tY%n", today);
  }
}
```

TIP

When using a date-related object to format multiple fields, as in this example, the `"1$"` (or appropriate parameter number, `"2$"`, `"3$"`, etc.) must be specified for each format code.

Running this `FormatterDates` class produces the following output:

```
$ java FormatterDates.java
2027-01-02
Today is January 02, 2027
$
```

String templates 21P

Java 21 and 22 introduced a preview feature `StringTemplate` class, which allowed one to interpolate variables into a string using `\{varName}`:

```
var str3 = STR."Hello \{name}, today is \{now}";
```

Sadly, Oracle withdrew the `StringTemplate` preview from Java 23, pending the creation of a better way of implementing string interpolation. The latest word on this is in JEP 465, which is now marked "closed/withdrawn." This JEP, like most, does have a good discussion of why the feature is needed and how they arrived at the design. Apparently the design did not pan out when used in "a substantial, real-world internal project" (Oracle's Gavin Bierman and Brian Goetz, *https://mail.openjdk.org/pipermail/amber-spec-experts/2024-April/004106.html*) so the feature was withdrawn until a better solution can be found.

3.3 Building Strings with StringBuilder

Problem

You need to put some `String` pieces (back) together.

Solution

Use string concatenation: the `+` operator. The compiler implicitly constructs a `StringBuilder` for you and uses its `append()` methods (unless all the string parts are known at compile time).

Better yet, construct and use a `StringBuilder` yourself.

Discussion

An object of the `StringBuilder` class basically represents a collection of characters. It is similar to a `String` object.[6] However, as mentioned, `String`s are immutable; `StringBuilder`s are mutable and designed for, well, building `String`s. You typically construct a `StringBuilder`, invoke the methods needed to get the character sequence just the way you want it, and then call `toString()` to generate a `String` representing the same character sequence for use in most of the Java API, which deals in `String`s.

A similar class, `StringBuffer`, is historical—it's been around since the beginning of time. Some of its methods are synchronized (see Recipe 11.4), which involves unneeded overhead in a single-threaded context. In Java 5, this class was split into `StringBuffer` (which is synchronized) and `StringBuilder` (which is not synchronized); thus, it is faster and preferable for single-threaded use. Another new class, `AbstractStringBuilder`, is the parent of both. In the following discussion, I'll use "the `StringBuilder` classes" to refer to all three because they mostly have the same methods.

The book's example code provides a `StringBuilderDemo` and a `StringBufferDemo`. Except for the fact that `StringBuilder` is not thread-safe, these API classes are identical and can be used interchangeably, so my two demo programs are almost identical except that each one uses the appropriate builder class.

The `StringBuilder` classes have a variety of methods for inserting, replacing, and otherwise modifying a given `StringBuilder`. Conveniently, the `append()` methods return a reference to the `StringBuilder` itself, so stacked statements like `.append(...).append(...)` are fairly common. This style of coding is referred to as a *fluent API* because it reads smoothly, like prose from a native speaker of a human language. You might even see this style of coding in a `toString()` method, for example. Example 3-3 shows three ways of concatenating strings.

Example 3-3. main/src/main/java/strings/StringBuilderDemo.java

```
public class StringBuilderDemo {

  public static void main(String[] argv) {

    String s1 = "Hello" + ", " + "World";
    System.out.println(s1);

    // Build a StringBuilder, and append some things to it.
    StringBuilder sb2 = new StringBuilder();
```

```
    sb2.append("Hello");
    sb2.append(',');
    sb2.append(' ');
    sb2.append("World");

    // Get the StringBuilder's value as a String, and print it.
    String s2 = sb2.toString();
    System.out.println(s2);

    // Now do the above all over again, but in a more
    // concise (and typical "real-world" Java) fashion.

    System.out.println(
      new StringBuilder()
      .append("Hello")
      .append(',')
      .append(' ')
      .append("World"));
  }
}
```

In fact, all the methods that modify the characters in a `StringBuilder` (i.e., `append()`, `delete()`, `deleteCharAt()`, `insert()`, `replace()`, and `reverse()`) return a reference to the builder object to facilitate this fluent API style of coding.

As another example of using a `StringBuilder`, consider the need to convert a list of items into a comma-separated list while avoiding getting an extra comma after the last element of the list. This can be done using a `StringBuilder`, although in Java 8+ there is a static `String` method to do the same. Code for these are shown in Example 3-4.

Example 3-4. main/src/main/java/strings/StringBuilderCommaList.java

```
    System.out.println(
      "Split using String.split; joined using 1.8 String join");
    System.out.println(String.join(", ", SAMPLE_STRING.split(" ")));

    System.out.println(
      "Split using String.split; joined using StringBuilder");
    StringBuilder sb1 = new StringBuilder();
    for (String word : SAMPLE_STRING.split(" ")) {
      if (sb1.length() > 0) {
        sb1.append(", ");
      }
      sb1.append(word);
```

```
    }
    System.out.println(sb1);
```

The first method is clearly the most compact; the static `String.join()` makes short work of this task. The next method uses the `StringBuilder.length()` method, so it will only work correctly when you are starting with an empty `StringBuilder`. A third method, in the online source but not printed here, involves a `StringTokenizer` and the information method `hasMoreElements()` in the `Enumeration`. An alternative method, particularly when you aren't starting with an empty builder, would be to use a `boolean` flag variable to track whether you're at the beginning of the list.

3.4 Processing a String One Character at a Time

Problem

You want to process the contents of a string, one character at a time.

Solution

Use a `for` loop and the `String`'s `charAt()` or `codePointAt()` method. Or use a `for each` loop and the `String`'s `toCharArray` method.

Discussion

A string's `charAt()` method retrieves a given character by index number (starting at zero) from within the `String` object. Since Unicode has had to expand beyond 16 bits, not all Unicode characters can fit into a Java `char` variable. There is thus an analogous `codePointAt()` method, whose return type is `int`. To process all the characters in a `String`, one after another, use a `for` loop ranging from zero to `String.length()-1`. Here we process all the characters in a `String`:

main/src/main/java/strings/strings/StrCharAt.java

```
public class StrCharAt {
   public static void main(String[] av) {
      String a = "A quick bronze fox";
   for (int i=0; i < a.length(); i++) { // no forEach, need the index
    String message =
      "charAt is '%c', codePointAt is %3d, casted it's '%c'".formatted(
```

```
        a.charAt(i),
        a.codePointAt(i),
        (char)a.codePointAt(i));
      System.out.println(message);
    }
  }
}
```

Given that the `for each` loop has been in the language for ages, you might be excused for expecting to be able to write some code like `for (char ch : myString) {...}`. Unfortunately, this does not work, as the `String` class is not iterable (maybe it should be, but it's complicated). You can use `myString.toCharArray()`, which does use some memory for the new array, as in the following:

```
public class ForEachChar {
  public static void main(String[] args) {
    String mesg = "Hello world";

    // Does not compile, Strings are not iterable
    // for (char ch : mesg) {
    //    System.out.println(ch);
    // }

    System.out.println("Using toCharArray:");
    for (char ch : mesg.toCharArray()) {
      System.out.println(ch);
    }

    System.out.println("Using Streams:");
    mesg.chars().forEach(c -> System.out.println((char)c));
  }
}
```

A *checksum* is a numeric quantity representing and confirming the contents of a file. If you transmit the checksum of a file separately from the contents, a recipient can checksum the file—assuming the algorithm is known—and verify that the file was received intact. Example 3-5 shows the simplest possible checksum, computed just by adding the numeric values of each character. Note that on files, it does not include the values of the newline characters; in order to fix this, retrieve `System.getProperty("line.separator");` and add its character value(s) into the sum at the end of each line. Or give up on line mode and read the file a character at a time.

Example 3-5. main/src/main/java/strings/CheckSum.java

```
/** Checksum one text file, given an open BufferedReader.
 * Checksum does not include line endings, so it will give the
 * same value for given text on any platform. Do not use
 * on binary files!
 */
public static int process(BufferedReader is) {
    int sum = 0;
    try {
        String inputLine;

        while ((inputLine = is.readLine()) != null) {
            for (char c : inputLine.toCharArray()) {
                sum += c;
            }
        }
    } catch (IOException e) {
        throw new RuntimeException("IOException: " + e);
    }
    return sum;
}
```

3.5 Aligning, Indenting, and Unindenting Strings

Problem

You want to align strings to the left, right, or center.

Solution

Do the math yourself, and use `substring` (see Recipe 3.1) and a `StringBuilder` (see Recipe 3.3). Or, use my `StringAlign` class, which is based on the `java.text.Format` class. For left or right alignment, use `String.format()`.

Discussion

Centering and aligning text comes up fairly often. Suppose you want to print a simple report with centered page numbers. There doesn't seem to be anything in the standard API

that will do the job fully for you. But I have written a class called `StringAlign` that will. Here's how you might use it:

```
public class StringAlignSimple {

  public static void main(String[] args) {
    // Construct a "formatter" to center strings.
    StringAlign formatter = new StringAlign(70, StringAlign.Justify.CENTER);
    // Try it out, for page "i"
    System.out.println(formatter.format("- i -"));
    // Try it out, for page 4. Since this formatter is
    // optimized for Strings, not specifically for page numbers,
    // we have to convert the number to a String
    System.out.println(formatter.format(Integer.toString(4)));
  }
}
```

If you run this class, it prints the two demonstration line numbers centered, as shown:

```
C:\> java StringAlignSimple.java
                                - i -
                                 4
C:\>
```

The method `String.repeat()` returns, as its name implies, a string that contains *n* copies of the input:

```
jshell> var response = "Aye ".repeat(2)
response ==> "Aye Aye "
jshell>
```

Example 3-6 is the code for my `StringAlign` class. Note that this class extends the class `Format` in the package java.text. There is a series of `Format` classes that all have at least one method called `format()`. It is thus in a family with numerous other formatters, such as `DateFormat` and `NumberFormat`, which we'll take a look at in upcoming chapters.

Example 3-6. main/src/main/java/strings/StringAlign.java

```
public class StringAlign extends Format {

 private static final long serialVersionUID = 1L;

 public enum Justify {
  /* Constant for left justification. */
  LEFT,
  /* Constant for centering. */
  CENTER,
  /** Constant for right-justified Strings. */
  RIGHT,
 }

 /** Current justification */
 private Justify just;
 /** Current max length */
 private int maxChars;

  /** Construct a StringAlign formatter; length and alignment are
   * passed to the Constructor instead of each format() call as the
   * expected common use is in repetitive formatting e.g., page numbers.
   * @param maxChars - the maximum length of the output
   * @param just - one of the enum values LEFT, CENTER, or RIGHT
   */
 public StringAlign(int maxChars, Justify just) {
  switch(just) {
  case LEFT:
  case CENTER:
  case RIGHT:
    this.just = just;
    break;
  default:
    throw new IllegalArgumentException("invalid justification arg.");
  }
  if (maxChars < 0) {
    throw new IllegalArgumentException("maxChars must be positive.");
  }
  this.maxChars = maxChars;
 }

 /** Format a String.
  * If the input is too long, it will be returned, truncated.
  * @param input - the string to be aligned.
  * @parm where - the StringBuilder to append it to.
```

```
 * @param ignore - a FieldPosition (may be null, not used but
 * specified by the general contract of Format).
 * @return A StringBufffer with the formatted string; sadly not
 * the newer StringBuilder because the old interface is cast in stone.
 */
@Override
public StringBuffer format(
  Object input, StringBuffer where, FieldPosition ignore)  {

  String s = input.toString();
  String wanted = s.substring(0, Math.min(s.length(), maxChars));

  // Get the spaces in the right place.
  switch (just) {
    case RIGHT:
      pad(where, maxChars - wanted.length());
      where.append(wanted);
      break;
    case CENTER:
      int toAdd = maxChars - wanted.length();
      pad(where, toAdd/2);
      where.append(wanted);
      pad(where, toAdd - toAdd/2);
      break;
    case LEFT:
      where.append(wanted);
      pad(where, maxChars - wanted.length());
      break;
    }
  return where;
}

protected final void pad(StringBuffer to, int howMany) {
  for (int i=0; i<howMany; i++)
    to.append(' ');
}

/** Convenience Routine */
String format(String s) {
  return format(s, new StringBuffer(), null).toString();
}

/** ParseObject is required, but not useful here. */
public Object parseObject(String source, ParsePosition pos)  {
  return source;
```

```
 }
}
```

A number of methods in `String` work on multiline strings, those with embedded newline characters. These can be produced in many ways, including multiline text strings (see Java 14 Text Blocks) or read from a file; for example, using `Files.readString()`. Java 12 introduced a new method, `public String indent(int n)`, that prepends *n* spaces to *each line* of the given multiline string. If *n* is negative, the number of spaces is removed from the front of each line. This works well in conjunction with the Java 11 `Stream<String> lines()` String instnce method. For example, for the case where a series of lines, already stored in a single string, needs the same indent (`Streams`, and the "::" notation, are explained in Recipe 9.8):

```
jshell> "abc\ndef".indent(30).lines().forEach(System.out::println);
                    abc
                    def

jshell> "abc\ndef".indent(30).indent(-10).lines().forEach(System.out::println);
              abc
              def

jshell>
```

`stripIndent()` finds and removes the smallest leading whitespace indentation in a multiline string, preserving relative indentation within the string:

```
jshell> var input = """
   ...>     The following:
   ...>       Volume 1
   ...>       Volume 4
   ...>         4A
   ...>         4B"""
input ==> "The following:\n   Volume 1\n   Volume 4\n     4A\n     4B"

jshell> System.out.println(input.stripIndent());
The following:
  Volume 1
  Volume 4
    4A
    4B
```

```
jshell>
```

In this example, four spaces have been removed from the front of each line.

See Also

The alignment of numeric columns is considered in Chapter 5.

3.6 Converting Between Unicode Characters and Strings

Problem

You want to convert between Unicode characters and `String`s.

Solution

Use Java `char` or `String` data types to deal with characters; these intrinsically support Unicode. Print characters as integers to display their *raw* value if needed.

Discussion

Unicode is an international standard that aims to represent all known characters used by people in all their various languages. Though the original ASCII character set is a subset, Unicode is huge. At the time Java was created, Unicode was a 16-bit character set, so it seemed natural to make Java `char` values be 16 bits in width, and for years a `char` could hold any Unicode character. However, over time, Unicode has grown, to the point that it now includes over a million code points, or characters, more than the 65,525 that could be represented in 16 bits.[7] Not all possible 16-bit values were defined as characters in UCS-2, the 16-bit version of Unicode originally used in Java. A few were reserved as escape characters, which allows for multicharacter-length mappings to less common characters. Fortunately, there is a go-between standard, called UTF-16 (16-bit Unicode Transformation Format). As the `String` class documentation puts it:

> *A `String` represents a string in the UTF-16 format in which supplementary characters are represented by surrogate pairs (see the section Unicode Character Representations in the `Character` class for more information). Index values refer to char code units, so a supplementary character uses two positions in a `String`.*
>
> *The `String` class provides methods for dealing with Unicode code points (i.e., characters), in addition to those for dealing with Unicode code units (i.e., `char values`).*

The `charAt()` method of `String` returns the `char` value for the character at the specified offset. If you might have to deal with character sets with characters beyond the 16-bit range, use `codePointAt()` instead. The `StringBuilder append()` method has a form that accepts a `char`. Because `char` is an integer type, you can even do arithmetic on `char`s, though this is not needed as frequently as in, say, C. Nor is it often recommended, because the `Character` class provides the methods for which these operations were normally used in languages such as C. Here is a program that uses arithmetic on `char`s to control a loop and that also appends the characters into a `StringBuilder` (see Recipe 3.3):

```
// UnicodeChars.java
StringBuilder b = new StringBuilder();
for (char c = 'a'; c<'d'; c++) {
 b.append(c);
}
b.append('\u00a5');  // Japanese Yen symbol
b.append('\u01FC');  // Roman AE with acute accent
b.append('\u0391');  // GREEK Capital Alpha
b.append('\u03A9');  // GREEK Capital Omega

for (int i=0; i<b.length(); i++) {
 System.out.printf(
  "Character #%d (%04x) is %c%n",
  i, (int)b.charAt(i), b.charAt(i));
}
System.out.println("Accumulated characters are " + b);
```

When you run it, the expected results are printed for the ASCII characters. On Unix and characters in Mac systems, the default fonts don't include all the additional characters, so they are either omitted or mapped to irregular characters:

```
$ java UnicodeChars.java
Character #0 (0061) is a
```

```
Character #1 (0062) is b
Character #2 (0063) is c
Character #3 (00a5) is ¥
Character #4 (01fc) is Ǽ
Character #5 (0391) is Α
Character #6 (03a9) is Ω
Accumulated characters are abc¥ǼΑΩ
$
```

The Windows system that I used to try this doesn't have most of those characters either, but at least it prints question marks for the ones it knows are lacking (Windows system fonts are more homogenous than those of the various Unix systems, so it may be easier to know what won't work):

```
$ java UnicodeChars.java
Character #0 is a
Character #1 is b
Character #2 is c
Character #3 is ¥
Character #4 is ?
Character #5 is ?
Character #6 is ?
Accumulated characters are abc¥...
```

The "." characters are unprintable characters.

Characters above `0xffff` need special handling; one way is to pass the numeric equivalents into the String constructor as an array of `int`s.

```
// Let's show Santa (0x1f385) + Star (0x2600):
int[] codePoints = {0x1f385, 0x2600};
String faceAndStar = new String(codePoints, 0, codePoints.length);
System.out.println(faceAndStar);    // "🎅☀"
```

See Also

The `Unicode` program in this book's online source displays any 256-character section of the Unicode character set. You can download documentation listing every character in the Unicode character set from the Unicode Consortium.

3.7 Reversing a String by Word or by Character

Problem

You wish to reverse a string, either character by character or word by word.

Solution

You can reverse a string by character easily, using a `StringBuilder`. There are several ways to reverse a string a word at a time. One natural way is to use a `StringTokenizer` and a stack. `Stack` is a class (defined in `java.util`; see Recipe 7.6) that implements an easy-to-use last-in, first-out (LIFO) stack of objects.

Discussion

To reverse the characters in a string, use the `StringBuilder reverse()` method:

main/src/main/java/strings/StringRevChar.java

```
String sh = "FCGDAEB";
System.out.println(sh + " -> " + new StringBuilder(sh).reverse());
```

The letters in this example list the order of the sharps in the key signatures of Western music; in reverse, it lists the order of flats. Alternatively, of course, you could reverse the characters yourself, using one-character-at-a-time mode (see Recipe 3.4).

A popular mnemonic, or memory aid, to help music students remember the order of sharps and flats consists of one word for each sharp instead of just one letter. Let's reverse this *one word at a time*. Example 3-7 adds each one to a `Stack` (see Recipe 7.6), then processes the whole lot in LIFO order, which reverses the order.

Example 3-7. main/src/main/java/strings/StringReverse.java

```
String sample = "Father Charles Goes Down And Ends Battle";

// An older way
// Put it in the stack frontward
Stack<String> myStack = new Stack<>();
var forward = sample.split("\\s");
```

```
for (String str : forward) {
  myStack.push(str);
}

// Print the stack backward
while (!myStack.empty()) {
  System.out.print(myStack.pop());
  System.out.print(' ');  // inter-word spacing
}
System.out.println();
```

This prints:

```
Battle Ends And Down Goes Charles Father
```

21 Java 21 added a default `reversed()` method to the `List` interface, and since Java 9 we could make a `List` from an array using an overload of `List.of()`. We can write this idiomatically in a single statement:

```
// The easier way (Java 21+, no temporary variables)
System.out.println(String.join(" ",
  List.of(sample.split("\\s")).reversed()));
```

This prints the list the same as Example 3-7. Note that `reversed()` creates a view onto the original list, so one needs to be careful about modifying the original while traversing the view.

3.8 Expanding and Compressing Tabs

Problem

You need to convert space characters to tab characters in a file, or vice versa. You might want to replace spaces with tabs to save space on disk or go the other way to deal with a device or program that can't handle tabs.

Solution

Use my `Tabs` class or its subclass `EnTab`.

Discussion

Because programs that deal with tabbed text or data expect tab stops to be at fixed positions, you cannot use a typical text editor to replace tabs with spaces or vice versa. Example 3-8 is a listing of `EnTab`, complete with a sample main program. The program works a line at a time. For each character on the line, if the character is a space, we see if we can coalesce it with previous spaces to output a single tab character. This program depends on the `Tabs` class, which we'll come to shortly. The `Tabs` class is used to decide which column positions represent tab stops and which do not.

Example 3-8. main/src/main/java/strings/Entab.java

```
public class EnTab {

 private static Logger logger = Logger.getLogger(EnTab.class.getSimpleName());

 /** The Tabs (tab logic handler) */
 protected Tabs tabs;

 /**
  * Delegate tab spacing information to tabs.
  */
 public int getTabSpacing() {
  return tabs.getTabSpacing();
 }

 /**
  * Main program: just create an EnTab object, and pass the standard input
  * or the named file(s) through it.
  */
 public static void main(String[] argv) throws IOException {
  EnTab et = new EnTab(8);
  if (argv.length == 0) // do standard input
   et.entab(
    new BufferedReader(new InputStreamReader(System.in)),
    System.out);
  else
   for (String fileName : argv) { // do each file
    et.entab(
     new BufferedReader(new FileReader(fileName)),
     System.out);
   }
 }
```

```
/**
 * Constructor: just save the tab values.
 * @param n The number of spaces each tab is to replace.
 */
public EnTab(int n) {
  tabs = new Tabs(n);
}

public EnTab() {
  tabs = new Tabs();
}

/**
 * entab: process one file, replacing blanks with tabs.
 * @param is A BufferedReader opened to the file to be read.
 * @param out a PrintWriter to send the output to.
 */
public void entab(BufferedReader is, PrintWriter out) throws IOException {

  // main loop: process entire file one line at a time.
  is.lines().forEach(line -> {
    out.println(entabLine(line));
  });
}

/**
 * entab: process one file, replacing blanks with tabs.
 *
 * @param is A BufferedReader opened to the file to be read.
 * @param out A PrintStream to write the output to.
 */
public void entab(BufferedReader is, PrintStream out) throws IOException {
  entab(is, new PrintWriter(out));
}

/**
 * entabLine: process one line, replacing blanks with tabs.
 * @param line the string to be processed
 */
public String entabLine(String line) {
  int N = line.length(), outCol = 0;
  StringBuilder sb = new StringBuilder();
  char ch;
  int consumedSpaces = 0;

  for (int inCol = 0; inCol < N; inCol++) { // Cannot use foreach here
```

```
      ch = line.charAt(inCol);
      // If we get a space, consume it, don't output it.
      // If this takes us to a tab stop, output a tab character.
      if (ch == ' ') {
        logger.info("Got space at " + inCol);
        if (tabs.isTabStop(inCol)) {
          logger.info("Got a Tab Stop " + inCol);
          sb.append('\t');
          outCol += consumedSpaces;
          consumedSpaces = 0;
        } else {
          consumedSpaces++;
        }
        continue;
      }

      // We're at a non-space; if we're just past a tab stop, we need
      // to put the "leftover" spaces back out, since we consumed
      // them above.
      while (inCol-1 > outCol) {
        logger.info("Padding space at " + inCol);
        sb.append(' ');
        outCol++;
      }

      // Now we have a plain character to output.
      sb.append(ch);
      outCol++;

    }
    // If line ended with trailing (or only!) spaces, preserve them.
    for (int i = 0; i < consumedSpaces; i++) {
      logger.info("Padding space at end # " + i);
      sb.append(' ');
    }
    return sb.toString();
  }
}
```

This code was patterned after a program in Brian Kernighan and P.J. Plauger's classic work *Software Tools* (Addison-Wesley). While their version was in a language called RatFor (Rational Fortran), my version has since been through several translations. Their version actually worked one character at a time, and for a long time I tried to preserve this overall structure. Eventually, I rewrote it to be a line-at-a-time program.

The program that goes in the opposite direction—taking tabs out instead of putting them in—is the `DeTab` class shown in Example 3-9; only the core methods are shown.

Example 3-9. main/src/main/java/strings/DeTab.java

```
public class DeTab {
 Tabs ts;

 public static void main(String[] argv) throws IOException {
  DeTab dt = new DeTab(8);
  dt.detab(new BufferedReader(new InputStreamReader(System.in)),
     new PrintWriter(System.out));
 }

 public DeTab(int n) {
  ts = new Tabs(n);
 }
 public DeTab() {
  ts = new Tabs();
 }

 /** detab one file (replace tabs with spaces)
  * @param is - the file to be processed
  * @param out - the updated file
  */
 public void detab(BufferedReader is, PrintWriter out) throws IOException {
  is.lines().forEach(line -> {
   out.println(detabLine(line));
  });
 }

 /** detab one line (replace tabs with spaces)
  * @param line - the line to be processed
  * @return the updated line
  */
 public String detabLine(String line) {
  char c;
  int col;
  StringBuilder sb = new StringBuilder();
  col = 0;
  for (int i = 0; i < line.length(); i++) {
   // Either ordinary character or tab.
   if ((c = line.charAt(i)) != '\t') {
    sb.append(c); // Ordinary
    ++col;
```

```
        continue;
      }
      do { // Tab, expand it, must put >=1 space
        sb.append(' ');
      } while (!ts.isTabStop(++col));
    }
    return sb.toString();
  }
}
```

The `Tabs` class maintains the tab settings and provides the `istabstop()` method. Example 3-10 is the source for the `Tabs` class.

Example 3-10. main/src/main/java/strings/Tabs.java

```
public class Tabs {
  /** tabs every so often */
  public final static int DEFTABSPACE =   8;
  /** the current tab stop setting. */
  protected int tabSpace = DEFTABSPACE;
  /** The longest line that we initially set tabs for. */
  public final static int MAXLINE  = 255;

  /** Construct a Tabs object with a given tab stop setting */
  public Tabs(int n) {
    if (n <= 0) {
      n = 1;
    }
    tabSpace = n;
  }

  /** Construct a Tabs object with a default tab stop setting */
  public Tabs() {
    this(DEFTABSPACE);
  }

  /**
   * @return Returns the tabSpace.
   */
  public int getTabSpacing() {
    return tabSpace;
  }

  /** isTabStop - returns true if given column is a tab stop.
   * @param col - the current column number
```

```
     */
    public boolean isTabStop(int col) {
      if (col <= 0)
        return false;
      return (col+1) % tabSpace == 0;
    }
  }
```

3.9 Controlling Case

Problem

You need to either convert strings to uppercase or lowercase or compare strings without regard for case.

Solution

The `String` class has a number of methods for dealing with documents in a particular case.

Discussion

String methods `toUpperCase()` and `toLowerCase()` each return a new string that is a copy of the current string but converted, as the name implies. Each can be called either with no arguments or with a `Locale` argument specifying the conversion rules; this is necessary because of internationalization. Java's API provides significant internationalization and localization features, as covered in "Ian's Basic Steps: Internationalization and Localization". Whereas the `equals()` method tells you if another string is exactly the same, `equalsIgnoreCase()` tells you if all characters are the same regardless of case. Here, you can't specify an alternative locale; the system's default locale is used, as in Example 3-11.

Example 3-11. main/src/main/java/strings/Case.java

```
    String name = "Java Cookbook";
    System.out.println("Normal:\t" + name);
    System.out.println("Upper:\t" + name.toUpperCase());
    System.out.println("Lower:\t" + name.toLowerCase());
    String javaName = "java cookBook"; // If it were Java identifiers :-)
    if (!name.equals(javaName))
```

```
    System.err.println("equals() correctly reports false");
  else
    System.err.println("equals() incorrectly reports true");
  if (name.equalsIgnoreCase(javaName))
    System.err.println("equalsIgnoreCase() correctly reports true");
  else
    System.err.println("equalsIgnoreCase() incorrectly reports false");
```

If you run this, it prints the first name changed to uppercase and lowercase, then it reports that both methods work as expected:

```
C:\javasrc\strings&gt;<strong>java Case.java</strong>
Normal: Java Cookbook
Upper:  JAVA COOKBOOK
Lower:  java cookbook
equals( ) correctly reports false
equalsIgnoreCase( ) correctly reports true
```

TIP

Java doesn't provide for uppercasing the first letter of each word (capitalization) of names, but Apache Commons Text does. Add the following dependency to your *pom.xml* or *build.gradle*:

```
org.apache.commons:commons-text:1.12.0
```

The Apache Commons `WordUtils` class provides a number of useful methods worth exploring in its documentation, but Example 3-12 includes examples of the capitalization methods.

Example 3-12. main/src/main/java/strings/Case.java

```
// Fancier stuff via Apache Commons Text:
String smith = "lorry smith";
System.out.println(WordUtils.capitalize(smith));
String jones = "JENkins JONes";
System.out.println(WordUtils.capitalizeFully(jones));
```

This segment prints:

```
Lorry Smith
Jenkins Jones
```

As illustrated, `capitalize()` just ensures the first letter of each word is capitalized, whereas `capitalizeFully()` also lowercases all subsequent letters in each word.

See Also

Regular expressions make it simpler to ignore case in string searching (as shown in Chapter 4).

3.10 Adding Nonprintable Characters into a String

Problem

You need to put nonprintable characters into strings.

Solution

Use the backslash character (`\`) to begin one of the Java string escapes.

Discussion

The Java string escapes are listed in Table 3-4.

Table 3-4. String escapes

To get	Use	Notes
Tab	`\t`	
Linefeed (Unix newline)	`\n`	The call `System.getProperty("line.separator")` will give you the platform's line end.
Carriage return	`\r`	
Form feed	`\f`	
Backspace	`\b`	
Single quote	`\'`	
Double quote	`\"`	
Unicode character	`\u` *NNNN*	Four hexadecimal digits (no `\x` as in C/C++). See the Unicode website for codes.
Octal(!) character	`\` *NNN*	Who uses octal (base 8) these days?
Backslash	`\\`	

Here is a code example that shows most of these in action:

```
public class StringEscapes {
  public static void main(String[] argv) {
    System.out.println("Java Strings in action:");
    System.out.println("A bell (alarm) entered in Octal: \007");
    System.out.println("A tab key: <<<\t>>>");
    System.out.println("A newline: <<<\n>>>");
    System.out.println("A UniCode character: \u0207");
    System.out.println("A backslash character: \\");
  }
}
```

```
A bell (alarm) entered in Octal:
A tab key: <<<      >>>
A newline: <<<
>>>
A UniCode character: ȇ
A backslash character: \
```

If you have a lot of non-ASCII characters to enter, you may want to consider using Java's input methods, discussed briefly in the infrequently updated Java 8 online documentation.

IAN'S BASIC STEPS: INTERNATIONALIZATION AND LOCALIZATION

Internationalization and localization consist of the following:

Sensitivity training (Internationalization, or I18N)

Making your software sensitive to these issues

Language lessons (Localization, or L10N)

Writing configuration files for each language

Culture lessons (optional)

Customizing the presentation of numbers, fractions, dates, and message formatting

For more information, see *Java Internationalization* by Andy Deitsch and David Czarnecki (O'Reilly).

3.11 Creating a Message to the World with I18N Resources

Problem

You want your program to take sensitivity training so that it can communicate well internationally.

Solution

Use internationalization software to obtain all strings to be displayed.

Discussion

Your program must obtain all control and message strings via the internationalization software. Here's how:

1. Get a `ResourceBundle` that has been saved on disk previously:

   ```
   ResourceBundle rb = ResourceBundle.getBundle("Menus");
   ```

   I'll cover `ResourceBundle` in Recipe 3.13, but briefly, a `ResourceBundle` represents a collection of name-value pairs (resources). The names are names you assign to each GUI control or other user interface text, and the values are the text to assign to each control in a given language.
2. Use this `ResourceBundle` to fetch the localized version of each control name.
   Old way:

   ```
   String label = "Exit";
   // Create the control, e.g., new JButton(label);
   ```

   New way:

   ```
   try { label = rb.getString("exit.label"); }
   catch (MissingResourceException e) { label="Exit"; } // fallback
   // Create the control, e.g., new JButton(label);
   ```

This may seem like quite a bit of code for one control, but you can write a convenience routine to simplify it, like this:

```
JButton exitButton = I18N.getButton(bundle, "exit.label", "Exit");
```

The file *I18N.java* is included in the book's code distribution (in *darwinsys-api*). There is also *I18NDemo.java* (in *javasrc/main/src/main/java/i18n/I18NDemo.java*).

While the example is a Swing `JButton`, the same approach goes with other UIs, such as the web tier.

What happens at runtime?

The default locale is used because we didn't specify one. The default locale is platform dependent:

Unix/POSIX

`LANG` environment variable (per user)

Windows

Control Panel → Regional Settings

macOS

System Preferences → Language & Text

Others

See platform documentation

`ResourceBundle.getBundle()` locates a file with the named resource bundle name (`Menus`, in the previous example), plus an underscore and the locale name (if a non-default locale is set), plus another underscore and the locale variation (if any variation is set), plus the extension *.properties*. If a variation is set but the file can't be found, it falls back to just the country code. If that can't be found, it falls back to the original default. Table 3-5 shows some examples for various locales.

Note that Android apps—usually written in Java or Kotlin—use a similar mechanism but with the files in XML format instead of Java Properties and with some small changes in the name of the file in which the properties files are found.

Table 3-5. Properties filenames for different locales

Locale	Filename
Default locale	*Menus.Properties*
Swedish	*Menus_sv.properties*
Spanish	*Menus_es.properties*
Cuban Spanish	*Menus_es_CU.properties*
French	*Menus_fr.properties*
French-Canadian	*Menus_fr_CA.properties*

Locale names are two-letter ISO 639 language codes (lowercase), and they normally abbreviate the country's *endonym* (the name its language speakers refer to it by); thus, Swedish is *sv* for *Sverige*, Spanish is *es* for *Español*, etc. Locale variations are two-letter ISO country codes (uppercase); for example, CA for Canada, US for the United States, SV for Sweden, ES for Spain, etc.

Setting the locale

On Windows, go into Regional Settings in the Control Panel. Changing this setting may entail a reboot, so exit any editor windows.

On Unix, set your `LANG` environment variable. For example, a Korn shell user in Mexico might have this line in their *.profile*:

```
export LANG=es_MX
```

On either system, for testing a different locale, you need only define the locale in the system properties at runtime using the command-line option `-D`, as in:

```
$ java -Duser.language=es Browser.java
```

This runs the Java program named `Browser` in package i18n in the Spanish locale.

You can get a list of the available locales with a call to `Locale.getAvailableLocales()`.

3.12 Using a Particular Locale

Problem

You want to use a locale other than the default in a particular operation.

Solution

Obtain a `Locale` by using a predefined instance or the `Locale` constructor. Optionally make it global to your application by using `Locale.setDefault(newLocale)`.

Discussion

Classes that provide formatting services, such as `DateTimeFormatter` and `NumberFormat`, provide overloads so they can be called either with or without a `Locale`-related argument.

To obtain a `Locale` object, you can employ one of the predefined locale variables provided by the `Locale` class, or you can construct your own `Locale` object giving a language code and a country code:

```
Locale locale1 = Locale.FRANCE;    // predefined
Locale locale2 = new Locale("en", "UK");    // English, UK version
```

These can then be used in the various formatting operations:

```
DateFormat frDateFormatter = DateFormat.getDateInstance(
        DateFormat.MEDIUM, frLocale);
DateFormat ukDateFormatter = DateFormat.getDateInstance(
        DateFormat.MEDIUM, ukLocale);
```

Either of these can be used to format a date or a number, as shown in Example 3-13.

Example 3-13. main/src/main/java/i18n/UseLocales.java

```
public class UseLocales {
  public static void main(String[] args) {

    Locale frLocale = Locale.FRANCE;  // predefined
    Locale ukLocale = Locale.of("en", "UK");  // English, UK version

    DateTimeFormatter defaultDateFormatter =
      DateTimeFormatter.ofLocalizedDateTime(
        FormatStyle.MEDIUM);
    DateTimeFormatter frDateFormatter =
      DateTimeFormatter.ofLocalizedDateTime(
        FormatStyle.MEDIUM).localizedBy(frLocale);
    DateTimeFormatter ukDateFormatter =
      DateTimeFormatter.ofLocalizedDateTime(
        FormatStyle.MEDIUM).localizedBy(ukLocale);

    LocalDateTime now = LocalDateTime.now();
```

```
        System.out.println("Default: " + ' ' +
          now.format(defaultDateFormatter));
        System.out.println(frLocale.getDisplayName() + ' ' +
          now.format(frDateFormatter));
        System.out.println(ukLocale.getDisplayName() + ' ' +
          now.format(ukDateFormatter));
      }
    }
```

The program prints the locale name and formats the date in each of the locales:

```
$ java UseLocales.java
Default:  Oct 16, 2025, 4:41:45 PM
French (France) 16 oct. 2025 à 16:41:45
English (UK) Oct 16, 2025, 4:41:45 PM
$
```

3.13 Creating a Resource Bundle

Problem

You need to create a resource bundle for use with I18N.

Solution

A resource bundle is simply a collection of names and values. You could write a `java.util.ResourceBundle` subclass, but it is easier to create textual `Properties` files (see Recipe 7.10) that you then load with `ResourceBundle.getBundle()`. The files can be created using any plain-text editor. Leaving it in a text file format also allows user customization in desktop applications; a user whose language is not provided for, or who wishes to change the wording somewhat due to local variations in dialect, should be able to edit the file.

Note that the resource bundle text file should not have the same name as any of your Java classes. The reason for this is that the `ResourceBundle` constructs a class dynamically with the same name as the resource files.

Discussion

Here is a sample properties file for a few menu items:

```
# Default Menu properties
# The File Menu
file.label=File Menu
file.new.label=New File
file.new.key=N
file.save.label=Save
file.new.key=S
```

Creating the default properties file is usually not a problem, but creating properties files for other languages might be. Unless you are a large multinational corporation, you will probably not have the resources (pardon the pun) to create resource files in-house. If you are shipping commercial software or using the web for global reach, you need to identify your target markets and understand which of these are most sensitive to wanting menus and the like in their own languages. Then, hire a professional translation service that has expertise in the required languages to prepare the files. Test them well—using native speakers of the target language, not affiliated with the translation house—before you ship, as you would any other part of your software.

If you need special characters, multiline text, or other complex entry, remember that a `ResourceBundle` is also a `Properties` file, so see the documentation for `java.util.Properties`.

3.14 Program: A Simple Text Formatter

This program is a primitive text formatter, representative of what people used on most computing platforms before the rise of standalone graphics-based word processors, laser printers, and, eventually, desktop publishing and office suites. It simply reads words from a file, previously created with a text editor, and outputs them until it reaches the right margin, when it calls `println()` to append a line ending. For example, here is an input file:

```
It's a nice
day, isn't it, Mr. Mxyzzptllxy?
I think we should go for a walk.
```

Given that file as the input, the `Fmt` program prints the lines formatted neatly:

```
It's a nice day, isn't it, Mr. Mxyzzptllxy? I think we should go for a walk.
```

As you can see, it fits the text we gave it to the margin and discards all single line breaks present in the original. The code is in the *javasrc* repo at *main/src/main/java/strings/Fmt.java* and online at my GitHub repository.

A slightly fancier version of this program, `Fmt2`, is also available at *include main/src/main/java/strings/Fmt2.java* or online at my GitHub repository.

The `Fmt2` version uses *dot commands*—lines beginning with periods—to give limited control over the formatting. A family of dot-command formatters includes Unix's `roff`, `nroff`, `troff`, and `groff`, which are in the same family with programs called `runoff` on Digital Equipment systems. The original program for this is believed to be J. Saltzer's `runoff`, which first appeared on Multics and from there made its way into various OSes. Incidentally, `Fmt2` subclasses `Fmt` and just overrides the `format()` method to include additional functionality.

1 The two `.equals()` calls are equivalent with the exception that the first can throw a `NullPointerException` while the second cannot.

2 `StringBuilder` was added in Java 5. It is functionally equivalent to the older `StringBuffer`. We will delve into the details in Recipe 3.3.

3 In addition to its use as a parameter in the `java.util.Formattable` interface, and not counting a dozen or so JDK-internal classes, `Formatter` is used by these classes: `java.net.HostPortrange`, `java.io.PrintWriter`, `java.io.PrintStream`, `java.lang.String`, and `java.util.logging.SimpleFormatter`.

4 Designed in the early 1970s by Mike Lesk at AT&T Bell Labs, it first saw widespread use in the library for the C language Dennis Ritchie had just invented, and it has been included or available in almost every programming language since. It has also, inevitably, grown significantly over time.

5 The central character in Yann Martel's novel *Life of Pi* would have been born in 1956, according to information in Wikipedia.

6 `String` and `StringBuilder` have several methods that are forced to be identical by their implementation of the `CharSequence` interface.

[7] Indeed, there are so many characters in Unicode that a fad has emerged of displaying your name upside down using characters that approximate upside-down versions of the Latin alphabet. Do a web search for “upside-down Unicode.”

Previous chapter

< 2. Software Development, Testing, and Maintenance

Next chapter

4. String Matching with Regular Expressions >